

PROGETTO OS/161 - PDSC2

Guida completa a system call, file descriptor, processi, fork/exec/wait, VFS ed eccezioni

Documento di studio per esposizione orale e revisione tecnica

Obiettivo del documento - Spiegare l'intero progetto senza dare per scontati i passaggi: che cosa richiedeva la specifica, quali strutture sono state aggiunte, dove si trovano le funzioni, come interagiscono, quali difficoltà sono emerse e come sono state risolte.

Mappa del progetto

Prima di entrare nei dettagli, conviene avere una mappa mentale dei file principali. Ogni file ha una responsabilit  precisa: alcuni definiscono strutture dati, altri implementano syscall, altri ancora gestiscono il passaggio user/kernel o il ciclo di vita dei processi.

File	Responsabilit� principale
kern/conf/PDSC2	Configurazione del kernel del progetto
kern/conf/conf.kern	Dichiarazione opzioni e inclusione dei sorgenti
kern/include/proc.h	Definizione di struct proc e campi aggiunti
kern/include/openfile.h	Definizione di struct openfile
kern/include/syscall.h	Prototipi delle syscall kernel
kern/syscall/file_syscalls.c	Open file table, open/read/write/close/lseek/dup2/chdir/getcwd
kern/syscall/proc_syscalls.c	getpid/fork/execv/waitpid/_exit
kern/proc/proc.c	Creazione/distruzione processi, PID table, file table, wait
kern/syscall/runprogram.c	Caricamento iniziale di un programma user
kern/arch/mips/syscall/syscall.c	Dispatcher MIPS e enter_forked_process
kern/arch/mips/locore/trap.c	Trap handling e kill_curthread
kern/arch/mips/vm/dumbvm.c	as_copy e gestione VM semplificata
kern/thread/thread.c	thread_exit e ciclo di vita del thread
kern/main/main.c	Bootstrap del kernel, incluso openfile_bootstrap

1. Configurazione del kernel PDSC2

OS/161 non compila automaticamente tutte le funzionalit  presenti nei sorgenti. Il kernel viene generato a partire da un file di configurazione che abilita opzioni specifiche. Per il progetto   stata quindi creata una configurazione dedicata, PDSC2.

```
kern/conf/PDSC2

options dumbvm
options syscalls
options synch
options waitpid
options dumbvm_with_free
options pdsc2
```

In kern/conf/conf.kern   stata dichiarata la nuova opzione con defoption pdsc2 e sono stati mantenuti/inclusi i sorgenti delle syscall del progetto, in particolare file_syscalls.c e proc_syscalls.c. In questo modo le parti del codice racchiuse da OPT_PDSC2 o dalle altre opzioni vengono effettivamente compilate solo nel kernel configurato per il progetto.

```
cd kern/conf
./config PDSC2
cd ../compile/PDSC2
bmake depend
bmake
bmake install
```

Da ricordare all'orale - PDSC2 non   una syscall:   la configurazione del kernel che abilita le funzionalit  del progetto. Le macro OPT_* derivano proprio da queste opzioni.

2. Dal programma user alla syscall kernel

Quando un programma user chiama una funzione come `read()`, `write()` o `fork()`, non entra direttamente nella nostra funzione `sys_read()` o `sys_fork()`. La libc user-level prepara i registri secondo la calling convention MIPS ed esegue l'istruzione `syscall`. Il processore genera una trap e il kernel salva lo stato della CPU in un `trapframe`.

```
PROGRAMMA USER
|
| read()/fork()/execv()
v
istruzione MIPS syscall
|
v
TRAP
|
v
struct trapframe
|
v
kern/arch/mips/syscall/syscall.c
|
v
sys_xxx()
```

Il `trapframe` è quindi una fotografia dello stato della CPU al momento dell'ingresso nel kernel. Tra i campi più importanti troviamo `v0`, `a0-a3`, `sp` ed `epc`. Nel dispatcher il numero della syscall viene letto da `tf->tf_v0`, mentre gli argomenti normalmente arrivano nei registri `a0-a3`.

Registro/campo	Significato nel contesto syscall
<code>v0</code>	Numero della syscall in ingresso; valore di ritorno in uscita
<code>a0-a3</code>	Primi quattro argomenti, salvo regole di allineamento
<code>a3</code>	Dopo la syscall vale 0 su successo, 1 su errore
<code>sp</code>	Stack pointer user; serve se gli argomenti non entrano nei registri
<code>epc</code>	Program counter da cui riprendere l'esecuzione user

La convenzione di ritorno è altrettanto importante: su successo `v0` contiene il risultato e `a3` viene impostato a 0; su errore `v0` contiene il codice `errno` e `a3` vale 1. La libreria user-level traduce poi questa rappresentazione nella convenzione C, tipicamente `return -1` e `errno` impostato.

```
SUCCESSO:  v0 = risultato, a3 = 0
ERRORE:     v0 = errno,    a3 = 1
```

Alla fine del dispatcher viene incrementato `tf->tf_epc` di 4 byte. L'istruzione `syscall` MIPS occupa infatti 4 byte: senza questo incremento, tornando in user mode il processore rieseguirebbe la stessa syscall all'infinito.

3. Architettura dei file: file descriptor, openfile e vnode

La scelta architetturale centrale della parte file è stata separare tre concetti che non devono essere confusi: il file descriptor, l'open file description e il vnode. Questa separazione è ciò che rende possibile implementare correttamente `close()`, `dup2()` e l'ereditarietà dei file aperti dopo `fork()`.

```
FILE_DESCRIPTOR != OPENFILE != VNODE

fd (numero del processo)
|
v
p_filetable[fd]
|
v
struct openfile
|
v
struct vnode
|
v
VFS / filesystem / device
```

3.1 File descriptor e File Descriptor Table

Il file descriptor è semplicemente un numero intero locale al processo. Ogni processo mantiene una propria file descriptor table, aggiunta a `struct proc` in `kern/include/proc.h`.

```
struct openfile *p_filetable[OPEN_MAX];
struct lock *p_filetable_lock;
```

Se il programma usa `fd = 3`, il kernel accede a `curproc->p_filetable[3]`. Il descriptor quindi non contiene direttamente il file, l'offset o i flag: è solo l'indice con cui il processo raggiunge un `openfile`.

3.2 struct openfile

La struttura `openfile` è definita in `kern/include/openfile.h`. Rappresenta una specifica apertura di un file, cioè lo stato che deve essere condiviso quando due descriptor derivano dalla stessa apertura.

```
struct openfile {
    struct vnode *of_vnode;
    off_t of_offset;
    int of_flags;
    unsigned int of_refcount;
    struct lock *of_lock;
};
```

Campo	Ruolo
<code>of_vnode</code>	Oggetto del VFS realmente aperto
<code>of_offset</code>	Posizione corrente condivisa nel file
<code>of_flags</code>	Modalità di apertura, ad esempio <code>O_RDONLY/O_WRONLY/O_APPEND</code>
<code>of_refcount</code>	Numero di riferimenti attivi verso quell' <code>openfile</code>
<code>of_lock</code>	Protegge lo stato condiviso, soprattutto l'offset

È importante notare che due chiamate separate a `open()` sullo stesso pathname possono produrre due `openfile` diversi e quindi due offset indipendenti. Al contrario, `dup2()` e `fork()` possono far condividere lo stesso `openfile`.

3.3 Vnode e VFS

Il vnode è l'oggetto astratto del Virtual File System. Le nostre syscall non accedono direttamente al disco: chiamano il VFS attraverso funzioni come `vfs_open()`, `vfs_close()`, `VOP_READ()`, `VOP_WRITE()`, `VOP_STAT()` e `VOP_ISSEEKABLE()`. In questo modo lo stesso codice può lavorare con file regolari, device e altri oggetti VFS.

3.4 System Open File Table

In `kern/syscall/file_syscalls.c` è stata introdotta una tabella globale di `openfile`. La File Descriptor Table è privata del singolo processo; la System Open File Table è invece una risorsa del kernel.

```
system_file_table[SYSTEM_OPEN_MAX]
system_file_table_lock
```

Questa distinzione consente di separare lo stato locale del processo dalla rappresentazione condivisibile dell'apertura del file.

4. Reference count e ciclo di vita di un openfile

Il reference count è necessario perché uno stesso `openfile` può essere raggiunto da più descriptor o da più processi. Il kernel deve chiudere realmente il vnode solo quando scompare l'ultimo riferimento.

```
fd 3 -----\
          >--- openfile X   (refcount = 2)
fd 7 -----/
```

Se chiudiamo `fd 3`, non possiamo chiamare subito `vfs_close()`, perché `fd 7` usa ancora lo stesso `openfile`. `openfile_release()` decrementa quindi `of_refcount` e libera le risorse solo quando il contatore raggiunge zero. `openfile_incref()` compie l'operazione opposta e viene usata quando nasce un nuovo riferimento, per esempio con `dup2()` o con l'ereditarietà della file table in `fork()`.

```
openfile_release(of)
|
+-- refcount > 0 -> openfile resta vivo
|
+-- refcount == 0 -> vfs_close(vnode)
                    lock_destroy(of_lock)
                    entry riutilizzabile
```

Domanda tipica - "Perché `close()` non chiude sempre il vnode?" Perché `close()` elimina un riferimento del processo. La chiusura reale avviene solo quando il reference count diventa zero.

5. Bootstrap degli openfile e descriptor standard

La funzione `openfile_bootstrap()`, implementata in `file_syscalls.c` e invocata durante il boot da `kern/main/main.c`, inizializza la System Open File Table e il lock globale prima che qualsiasi processo possa aprire file.

In `proc_create_runprogram()`, dentro `kern/proc/proc.c`, vengono inoltre creati i descriptor standard del nuovo processo. Questo è necessario affinché programmi user e shell possano usare subito input/output/error standard.

```
fd 0 -> con: aperta O_RDONLY (stdin)
fd 1 -> con: aperta O_WRONLY (stdout)
fd 2 -> con: aperta O_WRONLY (stderr)
```

Di conseguenza una `write(1, ...)` scrive sulla console, mentre una `read(0, ...)` legge dalla console.

6. `open()`, `close()`, `read()` e `write()`

6.1 `sys_open()` - `kern/syscall/file_syscalls.c`

`sys_open()` è il punto di ingresso kernel della syscall `open()`. Il `pathname` proviene dallo user space e viene quindi copiato con `copyinstr()`, che valida il puntatore e impedisce al kernel di dereferenziare direttamente memoria user potenzialmente non valida.

```
pathname user
|
v
copyinstr()
|
v
kernel_path
|
v
controllo flags
|
v
openfile_open()
|
v
vfs_open()
|
v
openfile
|
v
primo fd libero >= 3
```

La syscall distingue due errori importanti: `EMFILE` significa che il singolo processo ha esaurito i propri descriptor; `ENFILE` significa invece che la tabella globale degli `openfile` è piena.

6.2 `sys_close()` - `kern/syscall/file_syscalls.c`

`close()` verifica il range del descriptor, recupera l'`openfile` sotto `p_filetable_lock`, rimuove immediatamente l'entry dalla tabella del processo e poi chiama `openfile_release()`. Mettere prima l'entry a `NULL` rende il descriptor immediatamente invalido per il processo.

```
fd
|
v
p_filetable[fd] -> of
|
+--> p_filetable[fd] = NULL
|
+--> openfile_release(of)
```

6.3 sys_read() - kern/syscall/file_syscalls.c

read() trasferisce dati dal file verso un buffer user. Dopo aver validato fd e permessi di apertura, costruisce una iovec e una uio configurata con UIO_USERSPACE e UIO_READ. VOP_READ() esegue il trasferimento attraverso il VFS.

```
FILE / vnode
  |
  v
VOP_READ()
  |
  v
UIO_USERSPACE
  |
  v
buffer user
```

La iovec descrive dove si trova il buffer e quanto e grande. La uio descrive invece l'operazione completa: iovec, offset, residuo, direzione e address space. uio_resid indica quanti byte restano da trasferire: se chiediamo 100 byte e ne vengono letti 60, uio_resid sara 40 e il valore di ritorno sara $100 - 40 = 60$.

6.4 sys_write() - kern/syscall/file_syscalls.c

write() e simmetrica a read(): trasferisce dati dallo user buffer al vnode tramite una uio configurata con UIO_WRITE. Verifica che l'openfile sia stato aperto in O_WRONLY o O_RDWR e aggiorna l'offset condiviso usando il valore finale della uio.

Nel caso O_APPEND la posizione deve essere la fine attuale del file a ogni scrittura. Per questo viene chiamato VOP_STAT(), si legge st_size e si imposta l'offset alla fine prima di VOP_WRITE(). Non sarebbe sufficiente portare l'offset alla fine solo durante open(), perche il file puo crescere successivamente.

7. Concorrenza e lifetime dell'openfile

Durante lo sviluppo e emerso un problema piu sottile: non basta proteggere la sola lettura della file descriptor table. Se un thread recupera il puntatore a un openfile e rilascia p_filetable_lock, un altro thread potrebbe eseguire close() sullo stesso descriptor e portare il reference count a zero prima che il primo thread abbia terminato di usare l'oggetto.

```
THREAD A                                THREAD B

of = p_filetable[fd]
unlock filetable

                                     close(fd)
                                     refcount -> 0
                                     risorse openfile rilasciate

usa of  <-- rischio
```

La soluzione finale e prendere una reference temporanea con openfile_incref() mentre si e ancora protetti dalla file table, quindi rilasciare la filetable lock, eseguire read/write/lseek e infine chiamare openfile_release(). In questo modo l'openfile non puo scomparire mentre una syscall lo sta usando.

Decisione finale - Nelle note intermedie era stata valutata una soluzione piu semplice basata

solo sui lock. La versione finale aggiunge invece una reference temporanea per rendere sicura la lifetime dell'openfile.

8. lseek(): offset condiviso e calling convention a 64 bit

8.1 Logica di sys_lseek()

sys_lseek(), in kern/syscall/file_syscalls.c, modifica esclusivamente of_offset. Non legge ne scrive dati. Con SEEK_SET usa direttamente pos, con SEEK_CUR somma pos all'offset corrente, con SEEK_END usa VOP_STATO per ottenere la dimensione del file e calcola st_size + pos. Un offset finale negativo e invalido; inoltre la syscall verifica che il vnode sia seekable con VOP_ISSEEKABLE().

```
SEEK_SET -> newpos = pos
SEEK_CUR -> newpos = of_offset + pos
SEEK_END -> newpos = st_size + pos
```

8.2 Perché lseek è speciale in kern/arch/mips/syscall/syscall.c

off_t è un valore a 64 bit, mentre i registri MIPS della macchina usata da OS/161 sono a 32 bit. Inoltre gli argomenti a 64 bit devono iniziare su una coppia di registri allineata. La firma lseek(int fd, off_t pos, int whence) viene quindi distribuita in modo particolare.

```
a0 = fd
a1 = padding/allineamento
a2 = 32 bit alti di pos
a3 = 32 bit bassi di pos

whence -> non entra nei registri -> viene letto dallo user stack a sp + 16
```

Anche il valore restituito è a 64 bit e viene quindi suddiviso tra v0 e v1. Questo è stato verificato con /testbin/bigseek, che prova anche offset oltre i 32 bit.

9. dup2(), chdir() e __getcwd()

9.1 sys_dup2() - kern/syscall/file_syscalls.c

dup2(oldfd, newfd) non copia il file: fa semplicemente puntare newfd allo stesso openfile di oldfd. Se newfd era già aperto, il vecchio riferimento viene rilasciato. Il nuovo openfile riceve invece un reference count aggiuntivo.

```
PRIMA
fd 4 -> openfile A
fd 1 -> openfile B

DOPO dup2(4,1)
fd 4 --\
      >--- openfile A
fd 1 --/
```

Poiche i due descriptor puntano allo stesso openfile, condividono vnode, flags e soprattutto offset. Questa e la semantica necessaria per le redirezioni della shell. Il caso dup2(fd, fd) deve riuscire senza modificare nulla.

9.2 sys_chdir() - kern/syscall/file_syscalls.c

chdir() riceve un pathname user, lo copia in kernel space con copyinstr() e delega a vfs_chdir(). La syscall non manipola direttamente il filesystem: il VFS aggiorna la current working directory del processo.

9.3 sys___getcwd() - kern/syscall/file_syscalls.c

___getcwd() costruisce una uio verso memoria user e la passa a vfs_getcwd(). Il numero di byte trasferiti viene calcolato come buflen - uio_resid. Il test con pwd sulla root ha restituito emu0:, dimostrando che la catena user -> syscall -> VFS -> buffer user funziona. In una sottodirectory, pwd puo dipendere dal supporto VFS sottostante per ricostruire il pathname completo.

10. Architettura dei processi: struct proc e process table

Per implementare le syscall di processo e stato necessario estendere struct proc, definita in kern/include/proc.h e gestita in kern/proc/proc.c. Oltre a nome, thread, address space e cwd, il processo deve mantenere le informazioni necessarie alla gerarchia padre-figlio, al wait e ai file descriptor.

```
struct proc
|
+-- p_name
+-- p_numthreads
+-- p_addrspace
+-- p_cwd
+-- p_pid
+-- p_parent_pid
+-- p_status
+-- p_sem
+-- p_filetable[]
+-- p_filetable_lock
```

10.1 Process table e PID

In kern/proc/proc.c e stata introdotta una tabella globale che associa PID a struct proc. E protetta da spinlock per rendere atomiche le operazioni di allocazione e rimozione dei PID.

```
PID 1 -> proc A
PID 2 -> proc B
PID 3 -> proc C
```

proc_init_waitpid() assegna un PID e crea il semaforo del processo. proc_search_pid() cerca un processo a partire dal PID. proc_end_waitpid() rimuove il PID e distrugge il semaforo quando il processo viene definitivamente eliminato. Se la tabella e piena viene restituito ENPROC invece di causare un panic.

11. Creazione iniziale di un programma: proc_create_runprogram() e runprogram()

11.1 proc_create_runprogram() - kern/proc/proc.c

Questa funzione crea la struttura del processo che verrà usato per eseguire un programma avviato dal kernel menu. Il nuovo processo riceve un PID, eredita la current working directory, parte senza address space e ottiene i descriptor standard 0, 1 e 2 collegati a con:.

11.2 runprogram() - kern/syscall/runprogram.c

runprogram() si occupa invece del caricamento vero e proprio del programma. Apre l'eseguibile, crea un nuovo address space, lo installa nel processo, lo attiva, carica il file ELF, definisce lo stack e infine chiama enter_new_process().

```
vfs_open(programma)
|
v
as_create()
|
v
proc_setas(as) + as_activate()
|
v
load_elf()
|
v
as_define_stack()
|
v
enter_new_process()
```

La differenza fondamentale rispetto a `execv()` è che `runprogram()` parte da un processo senza address space e serve ad avviare inizialmente un programma; `execv()` parte invece da un processo già esistente e ne sostituisce il programma.

12. fork(): creazione del processo figlio

`sys_fork()` si trova in `kern/syscall/proc_syscalls.c`, ma coinvolge anche `kern/proc/proc.c`, `dumbvm.c`, `thread.c` e `syscall.c`. È una delle syscall più complesse perché deve creare un figlio che inizialmente rappresenti una copia quasi completa del padre.

```
PADRE
|
| fork()
v
proc_create_fork()
|-- nuovo PID
|-- parent_pid = PID padre
|-- eredita cwd
v
as_copy()
v
proc_filetable_copy()
v
copia trapframe
```

```
v
thread_fork()
v
FIGLIO
```

12.1 Cosa viene copiato e cosa viene condiviso

L'address space viene copiato con `as_copy()`. Con `dumbvm` si tratta di una vera copia fisica delle regioni e dello stack: dopo `fork` padre e figlio hanno memoria indipendente. La file descriptor table, invece, è distinta per i due processi ma le entry ereditate puntano agli stessi openfile, sui quali viene incrementato il reference count.

```
MEMORIA
PADRE address space --as_copy()--> FIGLIO address space indipendente

FILE
PADRE fd3 --\
                >--- stesso openfile
FIGLIO fd3 -/
```

Frase chiave - Dopo `fork`: address space copiato, openfile condivisi. La condivisione dell'openfile implica anche la condivisione dell'offset.

12.2 `proc_filetable_copy()` - `kern/proc/proc.c`

`proc_filetable_copy()` copia i puntatori presenti nella file table del padre nella tabella del figlio e chiama `openfile_incref()` su ogni entry ereditata. Il figlio ottiene quindi una propria tabella ma non nuove open file descriptions.

13. Trapframe e ritorno da `fork()`

Padre e figlio devono riprendere l'esecuzione dallo stesso punto, cioè dall'istruzione successiva alla `syscall fork()`, ma devono vedere valori di ritorno diversi. Il padre riceve il PID del figlio; il figlio deve ricevere 0.

Il trapframe del padre viene copiato in memoria kernel e passato al thread del figlio. La funzione `enter_forked_process()`, in `kern/arch/mips/syscall/syscall.c`, copia il trapframe sul kernel stack del figlio, libera la copia temporanea e modifica i registri di ritorno.

```
child_tf.tf_v0 = 0;    // fork() ritorna 0 nel figlio
child_tf.tf_a3 = 0;    // successo
child_tf.tf_epc += 4;  // salta l'istruzione syscall

as_activate();
mips_usermode(&child_tf);
```

`as_activate()` rende attivo l'address space del figlio, mentre `mips_usermode()` ripristina i registri e trasferisce definitivamente l'esecuzione in user mode.

14. execv(): sostituzione del programma corrente

sys_execv(), in kern/syscall/proc_syscalls.c, non crea un nuovo processo. Mantiene identità e risorse del processo - quindi PID, parent PID, cwd e file descriptor - ma sostituisce completamente address space, codice, dati e stack con quelli del nuovo eseguibile.

```
PRIMA: PID 5 -> programma A
execv(...)
DOPO:  PID 5 -> programma B
```

```
Restano: PID, parent PID, cwd, file descriptor
Cambiano: address space, codice, dati, stack
```

14.1 Perché argv è difficile

argv è un array in memoria user che contiene puntatori user a stringhe user. Non può essere usato direttamente dal kernel. Per ogni elemento si usa copyin() per leggere il puntatore e poi copyinstr() per copiare la stringa.

```
argv (user array)
|
+-- user pointer -> "arg0"
+-- user pointer -> "arg1"
+-- user pointer -> "arg2"
+-- NULL

copyin()    -> legge il puntatore
copyinstr() -> legge la stringa puntata
```

14.2 Sequenza completa di execv()

```
programe + argv user
|
v
copyinstr(programe)
|
v
scansione argv + copyin/copyinstr
|
v
controllo ARG_MAX
|
v
vfs_open(eseguibile)
|
v
as_create()
|
v
proc_setas(newas) + as_activate()
|
v
load_elf()
|
v
as_define_stack()
|
v
copia stringhe argv sul nuovo stack user
|
v
```

```

costruzione array argv[]
|
v
distruzione old address space
|
v
enter_new_process()

```

Un aspetto importante è la gestione degli errori: il vecchio address space viene distrutto solo quando il nuovo programma è completamente pronto. Se `load_elf()`, la definizione dello stack o un copyout falliscono, il codice ripristina il vecchio address space e fa fallire `execv()` senza distruggere il processo corrente.

14.3 Stack e ARG_MAX

Lo stack user cresce verso indirizzi inferiori. Le stringhe vengono quindi copiate a partire dall'ultimo argomento, rispettando l'allineamento, e successivamente viene copiato l'array dei puntatori `argv[]`. Ogni elemento di `argv` deve contenere un indirizzo user valido e `argv[argc]` deve essere NULL.

```

USERSTACK (indirizzi alti)
|
v
"ultimo argomento\0"
"... \0"
"nome programma\0"
argv[0..argc] <-- argv[argc] = NULL
|
v
indirizzi più bassi

```

Il controllo ARG_MAX considera non solo il numero dei puntatori, ma la dimensione complessiva delle stringhe più l'array `argv[]`. Il test `/testbin/argtest` uno due tre ha confermato la correttezza di `argc`, `argv` e del terminatore NULL.

15. getpid(), _exit() e waitpid()

15.1 getpid() - kern/syscall/proc_syscalls.c

`getpid()` è la syscall più semplice: restituisce `curproc->p_pid`. Il PID viene assegnato durante la creazione del processo e permette di identificare il processo nella process table.

15.2 _exit() - kern/syscall/proc_syscalls.c

`_exit(status)` termina normalmente il processo. Lo status non viene salvato come semplice intero, ma codificato con `_MKWAIT_EXIT(status)` nel formato atteso da `waitpid()`. L'address space viene scollegato dal processo e distrutto, perché non verrà più eseguito codice user. La struct `proc`, invece, deve rimanere viva perché il padre deve ancora recuperare PID e status.

```

_exit(status)
|
v
p_status = _MKWAIT_EXIT(status)
|
v
proc_setas(NULL) + as_destroy(oldsas)

```

```

|
v
V(p_sem)
|
v
thread_exit()

```

15.3 waitpid() - kern/syscall/proc_syscalls.c e proc_wait() in kern/proc/proc.c

waitpid() verifica le opzioni, cerca il processo tramite PID, controlla la relazione padre-figlio e poi chiama proc_wait(). proc_wait() esegue P(child->p_sem). Se il figlio non è ancora terminato, il padre va in sleep; quando il figlio esegue V(p_sem) in _exit(), il padre viene risvegliato.

PADRE	FIGLIO
waitpid(child)	
P(child->p_sem)	
sleep	
	_exit(status)
	p_status = ...
	V(p_sem)
<-----	
wake up	
legge status	
proc_destroy(child)	
ritorna PID	

Il semaforo serve quindi a sincronizzare un evento, non semplicemente a proteggere una struttura. Evita il busy waiting che avremmo con un ciclo continuo di controllo dello stato del figlio.

16. Relazione tra _exit(), thread_exit() e proc_remthread()

Durante il debug era stato inizialmente inserito proc_remthread(curthread) direttamente dentro _exit(). Controllando kern/thread/thread.c è emerso però che thread_exit() esegue già il detach del thread dal processo. Lasciare entrambe le chiamate avrebbe provocato un doppio detach e possibili KASSERT.

```

CORRETTO
sys__exit()
|
v
thread_exit()
|
v
proc_remthread()

SBAGLIATO
sys__exit() -> proc_remthread() -> thread_exit() -> proc_remthread() di nuovo

```

Lezione architetturale - Nel kernel è fondamentale capire chi possiede la responsabilità di rilasciare una risorsa. Due funzioni che fanno la stessa cleanup possono essere più pericolose di una funzione che la dimentica.

17. kill_curthread(): fatal exception user senza kernel panic

kill_curthread() si trova in kern/arch/mips/locore/trap.c. Nello skeleton originale una fatal exception user terminava con un panic del kernel. La specifica del progetto richiedeva invece che un errore di un programma user terminasse solo quel processo.

La funzione traduce il codice di trap in un segnale Unix coerente, ad esempio SIGSEGV per accessi di memoria invalidi, SIGILL per istruzioni illegali e SIGFPE per overflow aritmetici. Lo status viene codificato con _MKWAIT_SIG(sig), l'address space viene distrutto, il padre eventualmente in attesa viene svegliato con V(p_sem) e il thread termina con thread_exit().

```
fatal user trap
|
v
trap code -> signal
|
v
p_status = _MKWAIT_SIG(sig)
|
v
destroy address space
|
v
V(p_sem)
|
v
thread_exit()
|
v
KERNEL CONTINUA
```

Il test /testbin/faulter ha verificato proprio questo comportamento: il programma user viene terminato ma OS/161 torna al menu senza panic.

18. Lock, spinlock e semafori: perche ne servono tre tipi

Il progetto usa primitive di sincronizzazione diverse per problemi diversi. Non sono intercambiabili e il professore potrebbe chiedere perche una struttura usa un lock e un'altra un semaforo.

Primitiva	Dove viene usata	Perche
Spinlock	Process table, p_lock	Sezioni critiche brevissime; non deve dormire
Lock	p_filetable, system open file table, openfile state	Mutua esclusione su strutture che possono richiedere operazioni piu lunghe
Semaphore	waitpid <-> _exit	Sincronizzazione di un evento; il padre deve dormire finche il figlio non termina

p_filetable_lock protegge la mapping fd -> openfile. of_lock protegge lo stato dell'openfile, soprattutto of_offset, che puo essere condiviso. system_file_table_lock protegge allocazione, rilascio e reference count nella tabella globale. La process table usa uno spinlock per rendere atomiche le operazioni sui PID.

19. proc_destroy(): rilascio definitivo delle risorse

proc_destroy(), in kern/proc/proc.c, viene chiamata quando una struct proc può essere eliminata definitivamente. Deve liberare tutte le risorse ancora possedute dal processo, in un ordine coerente.

```
proc_destroy()
|
|-- release cwd (VOP_DECREF)
|-- destroy address space se ancora presente
|-- rimuove tutti i fd
|-- openfile_release() per ogni openfile
|-- destroy p_filetable_lock
|-- rimuove PID dalla process table
|-- destroy semaphore
|-- cleanup p_lock
|-- free nome
|-- free struct proc
```

Per un figlio terminato la struct proc non viene eliminata immediatamente da _exit(): rimane disponibile finché il padre non raccoglie lo status con waitpid(). Questo comportamento è concettualmente simile a uno stato zombie minimale.

20. Difficoltà reali incontrate e come sono state risolte

20.1 Race tra shell e kernel menu sulla console

Uno dei bug più fuorvianti era il fatto che un comando digitato nella shell, ad esempio pwd, venisse ricevuto mutilato. Sembrava un errore di read(), fork() o execv(), ma la causa era che il kernel menu e /bin/sh leggevano contemporaneamente dallo stesso device con:. I caratteri venivano quindi consumati da due reader diversi.

```
INPUT: p  w  d
      |  |  |
      |  |  +--> shell
      |  +-----> kernel menu
      +-----> shell
```

La soluzione è stata fare in modo che il kernel menu aspetti il completamento del programma user prima di tornare a leggere dalla console. La lezione è importante: il sintomo appariva nella shell, ma la causa era una race di I/O tra due componenti.

20.2 ENOMEM durante fork() con dumbvm

Un altro problema era fork: Out of memory. Il sistema era configurato con poca RAM e utilizzava dumbvm. In dumbvm as_copy() effettua una copia fisica completa dell'address space, senza copy-on-write. Durante una sequenza shell -> fork -> execv possono quindi coesistere il padre, la copia del figlio e temporaneamente anche il nuovo address space creato da execv().

```
PADRE address space
+
FIGLIO address space copiato da as_copy()
+
NUOVO address space temporaneo durante execv()
=
picco di memoria elevato
```

Aumentando la RAM configurata in System/161 il problema è scomparso, dimostrando che non si trattava necessariamente di un bug di fork() ma anche di un limite della VM semplificata.

20.3 ARG_MAX e undefined symbols

Durante l'implementazione di execv() erano inizialmente undefined simboli come ARG_MAX e PATH_MAX. La causa era l'assenza dell'header corretto; includendo limits.h si sono rese disponibili le costanti usate per limitare pathname e argomenti.

20.4 as_activate() e address space

Un altro punto delicato è stato capire quando chiamare as_activate(). La funzione deve essere usata dopo aver installato l'address space corretto nel processo, per rendere coerente la vista della CPU/TLB con curproc. In fork viene chiamata prima di mips_usermode() nel figlio; in execv viene chiamata dopo proc_setas(newas) e quando si ripristina oldas in caso di errore.

20.5 getcwd in sottodirectory

Il test pwd sulla root ha funzionato, mentre in una sottodirectory il VFS poteva rispondere Function not implemented. Poiché chdir() riusciva e badcall validava la syscall, non è stata modificata arbitrariamente sys___getcwd(): il limite dipendeva dal supporto VFS sottostante per ricostruire il pathname completo.

21. Error handling: cosa deve succedere quando l'utente sbaglia

La specifica non richiedeva soltanto il percorso normale. Le syscall devono rifiutare parametri invalidi senza causare kernel panic. Il dispatcher restituisce il codice errno in v0 e pone a3 = 1.

Errore	Significato tipico nel progetto
EBADF	File descriptor invalido, chiuso o con access mode incompatibile
EFAULT	Puntatore user invalido
EINVAL	Argomento semanticamente invalido, ad esempio options/whence
ENOENT	File o pathname inesistente
EMFILE	File descriptor table del processo piena
ENFILE	System Open File Table piena
ENOMEM	Memoria kernel/fisica insufficiente
ENPROC	Process table senza PID liberi
ECHILD	waitpid su un processo che non è figlio
ESRCH	PID non trovato
ESPIPE	lseek su oggetto non seekable
E2BIG	Argomenti execv oltre ARG_MAX

Il programma /testbin/badcall è stato particolarmente importante perché passa intenzionalmente fd negativi, puntatori NULL o non validi, PID inesistenti, whence errati e altri casi anomali. Il comportamento corretto è restituire errno e mantenere il kernel operativo.

22. Batteria di test finale e cosa dimostra ogni test

Test	Cosa verifica
/testbin/faulter	kill_curtread e gestione fatal user trap senza kernel panic
/testbin/bigseek	lseek a 64 bit, offset grandi, SEEK_*
/testbin/redirect	dup2 e redirezione dei descriptor
/testbin/forktest	fork e creazione concorrente di processi
/testbin/argtest uno due tre	execv, argc/argv, stack e terminatore NULL
/testbin/badcall	Error handling delle syscall richieste
/bin/sh + true/false	Catena fork -> execv -> _exit -> waitpid
cat /bin/true	open/read/write/close su file reale
cp /bin/true prova	Creazione, lettura e scrittura file
pwd/cd	chdir e getcwd/VFS

Il caso false -> Exit 1 e un test integrato particolarmente significativo: la shell fa fork, il figlio esegue execv del programma false, il programma termina con _exit(1), il padre esegue waitpid e interpreta correttamente lo status.

23. Syscall non implementate e perche

Alcuni test usavano anche syscall accessorie come remove() o fstat(). In alcuni casi compariva quindi Unknown syscall, pur dopo che il test principale aveva già stampato Passed. Queste syscall non sono state aggiunte perché non facevano parte dello scope richiesto dal progetto. È una scelta corretta evitare di ampliare inutilmente l'implementazione introducendo funzionalità non richieste e potenziali regressioni.

24. Domande probabili del professore e risposte tecniche

Cos'è un file descriptor?

È un indice nella file descriptor table privata del processo. L'entry punta a un openfile; il descriptor non contiene direttamente vnode, offset o flags.

Cos'è un openfile?

È la rappresentazione kernel di una specifica apertura di un file. Contiene vnode, offset, flags, reference count e lock.

Perché serve il reference count?

Perché lo stesso openfile può essere condiviso da più descriptor o processi. Il vnode viene chiuso solo quando scompare l'ultimo riferimento.

Cosa condivide fork()??

L'address space viene copiato; la file descriptor table è distinta, ma gli openfile ereditati sono condivisi e quindi condividono anche l'offset.

Cosa fa `execv()`?

Non crea un processo nuovo. Mantiene PID, parent PID, cwd e file descriptor, ma sostituisce address space, programma e stack.

Perche serve il trapframe in `fork()`?

Per salvare lo stato CPU del padre e far ripartire il figlio dallo stesso punto, impostando pero il return value della fork a 0.

Come sincronizzi `waitpid` e `_exit`?

Con un semaforo per-processo: il padre esegue P(), il figlio in `_exit` esegue V(). Il padre dorme senza busy waiting.

Perche `_exit` non distrugge subito `struct proc`?

Perche il padre deve ancora recuperare PID e status tramite `waitpid`.

Perche `lseek` e speciale?

Perche `off_t` e a 64 bit su MIPS a 32 bit: pos usa `a2/a3`, whence viene letto dallo stack e il risultato usa `v0/v1`.

Perche `copyin/copyout`?

Perche il kernel non deve dereferenziare direttamente puntatori user, che possono essere invalidi o puntare fuori dallo spazio consentito.

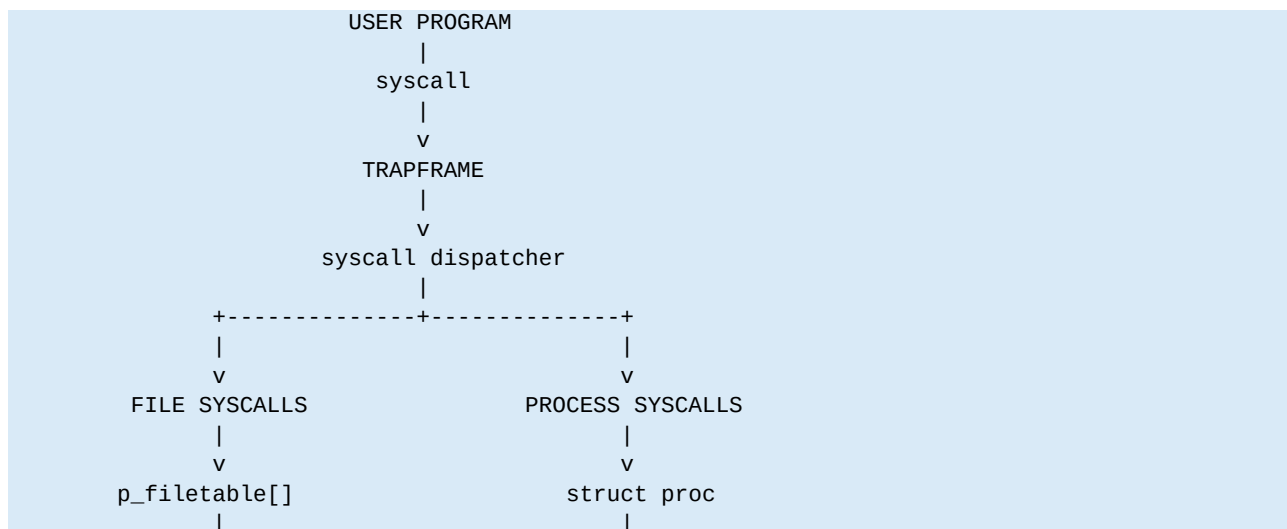
Cosa fa `kill_curthread`?

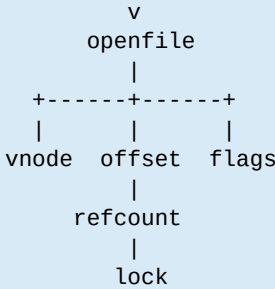
Converte la fatal trap user in uno status da segnale e termina solo il processo user, lasciando il kernel operativo.

Quali sono state le difficolta principali?

Trapframe e semantica di `fork`; `argv` e stack di `execv`; condivisione/lifetime degli `openfile`; race shell/menu sulla console; `ENOMEM` dovuto a `dumbvm`.

25. Mappa mentale finale





L'idea piu importante con cui chiudere il progetto e questa: la syscall e solo il punto di ingresso. La correttezza nasce dalle strutture kernel che mantengono lo stato nel tempo - openfile, file table, process table, address space, trapframe e primitive di sincronizzazione - e dal modo in cui queste strutture vengono condivise, copiate e distrutte.

26. Checklist finale da ripetere prima dell'orale

- File descriptor = indice per-processo; openfile = stato condivisibile; vnode = oggetto VFS.
- open() crea/openfile e assegna fd; close() elimina un riferimento; refcount decide la chiusura reale.
- read/write usano iovec + uio + VOP_READ/VOP_WRITE e aggiornano l'offset dell'openfile.
- dup2() fa condividere lo stesso openfile; quindi condivide anche l'offset.
- lseek() modifica solo l'offset; off_t a 64 bit richiede gestione speciale nel dispatcher MIPS.
- fork() crea nuovo PID, copia address space, eredita cwd, condivide openfile e copia trapframe.
- Il padre vede PID figlio, il figlio vede 0: enter_forked_process modifica il trapframe.
- execv() non crea un nuovo processo: sostituisce programma/address space e ricostruisce argv sul nuovo stack.
- waitpid e _exit usano un semaforo; la struct proc del figlio sopravvive finche lo status non viene raccolto.
- thread_exit() esegue gia proc_renthread(): non va duplicato in _exit().
- kill_curthread termina il processo user, non il kernel.
- Le difficoltà più indicative sono race console, lifetime openfile e memoria limitata di dumbvm.
- badcall dimostra che gli errori vengono gestiti con errno invece che con panic.